

Sreeraaghav Raja
2/18/26
40990068

EE Design 1 Technical Report

Microcontroller ADC & LCD

Introduction

This lab was focused on learning how to use an Analog-to-Digital Convertor (ADC) to properly read analog values from a resistor ladder and print them out to a Liquid Crystal Display (LCD). However, we could not use any communication program to directly connect to the LCD, so we had to emulate our own communication protocol through a unique method. Moreover, this method made the resistance values unreliable, so we had to change the design until the output resistance values met the specifications.

Methods

Background Information:

The key to analyzing the ADC is to first understand why we require them. The most fundamental issue with any embedded device is that it struggles to read continuous values. The main challenge with digital devices is that they struggle to discretize continuous signals and the ADC is a fundamental asset in smoothing out this process.

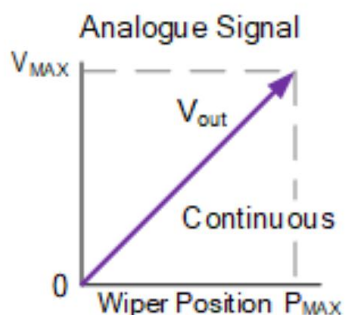


Figure 1: Potentiometer Circuit V_{out} Reading

Figure 1 is an example of the change in the voltage reading at the output node of a potentiometer [1]. The reading has no breaks in between any of the wiper positions. It is difficult to utilize any number of bits to perfectly match an analog value, so ADCs often have a **resolution** that is based on the number of bits that it can utilize. For instance, a 12-bit ADC will have a resolution of $2^{12} - 1 = 4095$, so the numbers it can read can range from 0 to 4095, unless they are normalized. Moreover, the resolution of the ADC corresponds to the number of steps that an ADC has between its entire range [1].

Figure 2 demonstrates the “same” functionality as the original potentiometer circuit but with a **rotary switch**. A rotary switch operates as complex switch that has multiple position besides

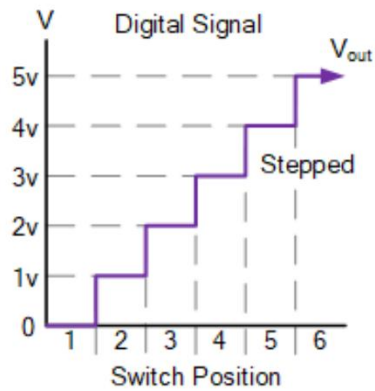


Figure 2: Rotary Switch Circuit Vout Reading

resolution will result in greater accuracy. For instance, a 5V signal with a 5-bit ADC will have a span of 5V, a resolution of 32, and an accuracy of 156.25mV; the calculation is demonstrated by the following equation:

$$Accuracy = \frac{Span}{Resolution} = \frac{5}{32} = 0.15625$$

the binary “On” and “Off” positions of a regular switch [2]. Figure 2 differs from Figure 1 based on the different readings for the same range of outputs. The only way to digitally replicate the analog *wiper position* is to have each rotary switch position act as *bin*, a range of values, for a certain amount of wiper positions. Therefore, **V_{out}** becomes discretized into 6 distinct voltages instead of a complete range of continuous values.

However, the issue of accuracy is trivial if we take the resolution into consideration. Accuracy is equivalent to **span / resolution**. **Span** is the range of voltage values that we are digitizing. Therefore, a smaller span and larger

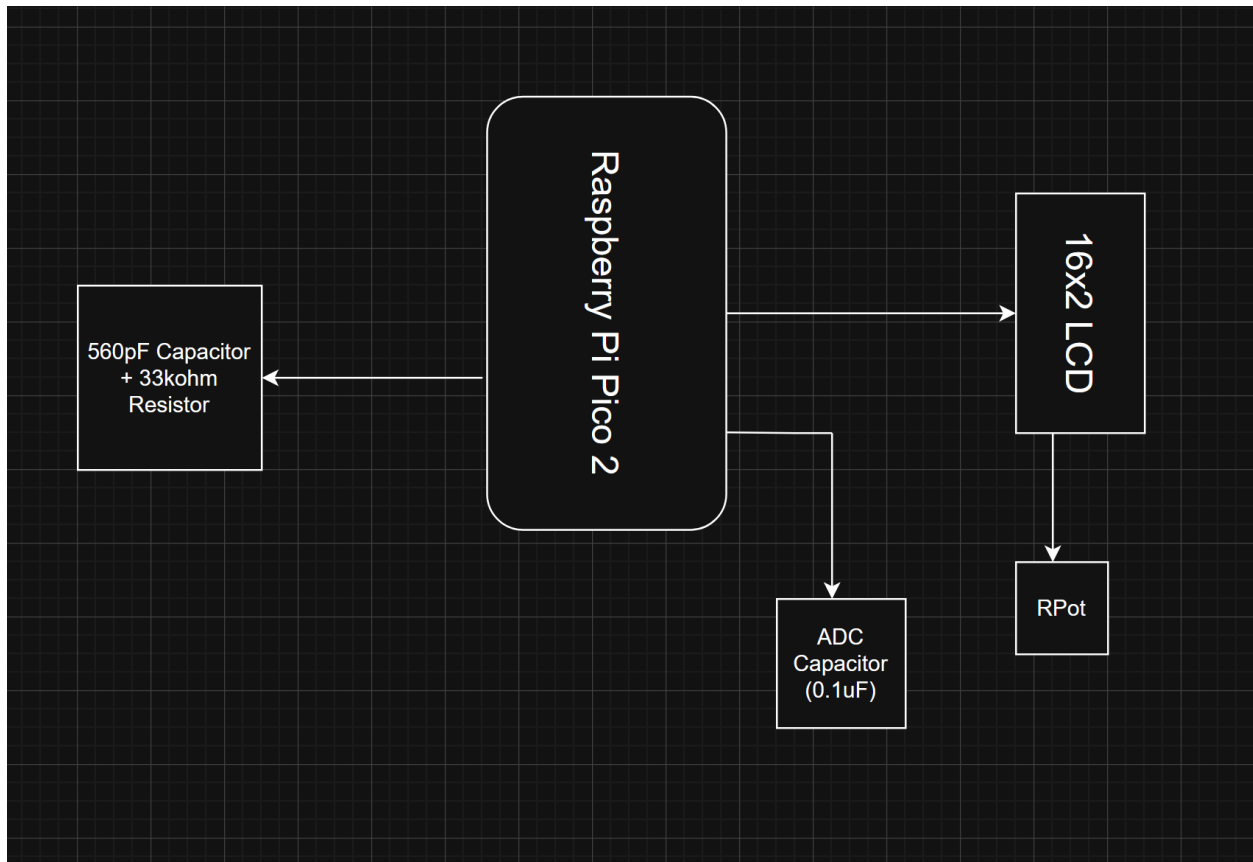


Figure 3: Hardware Block Diagram

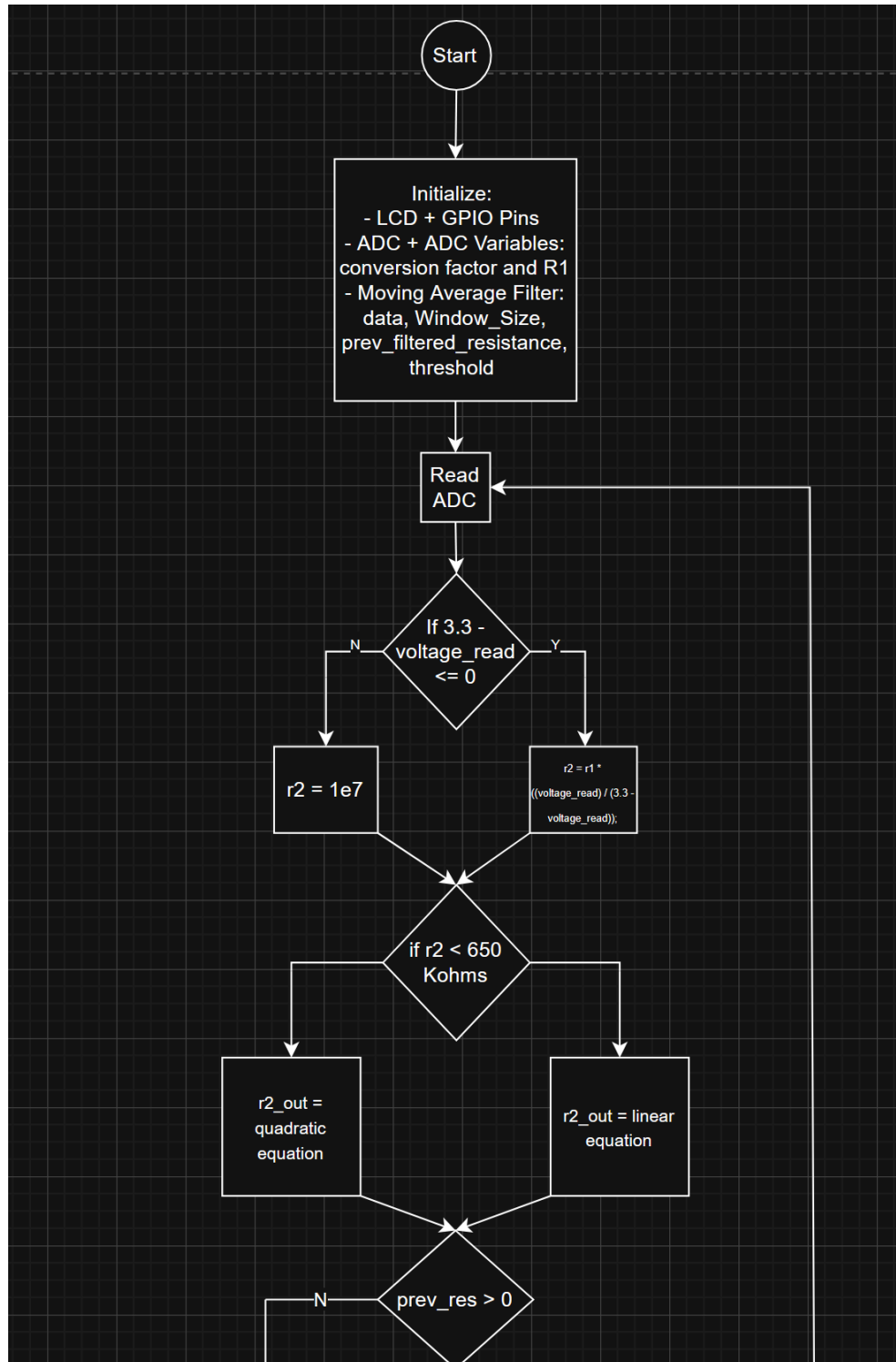


Figure 4: SW Block Top Half

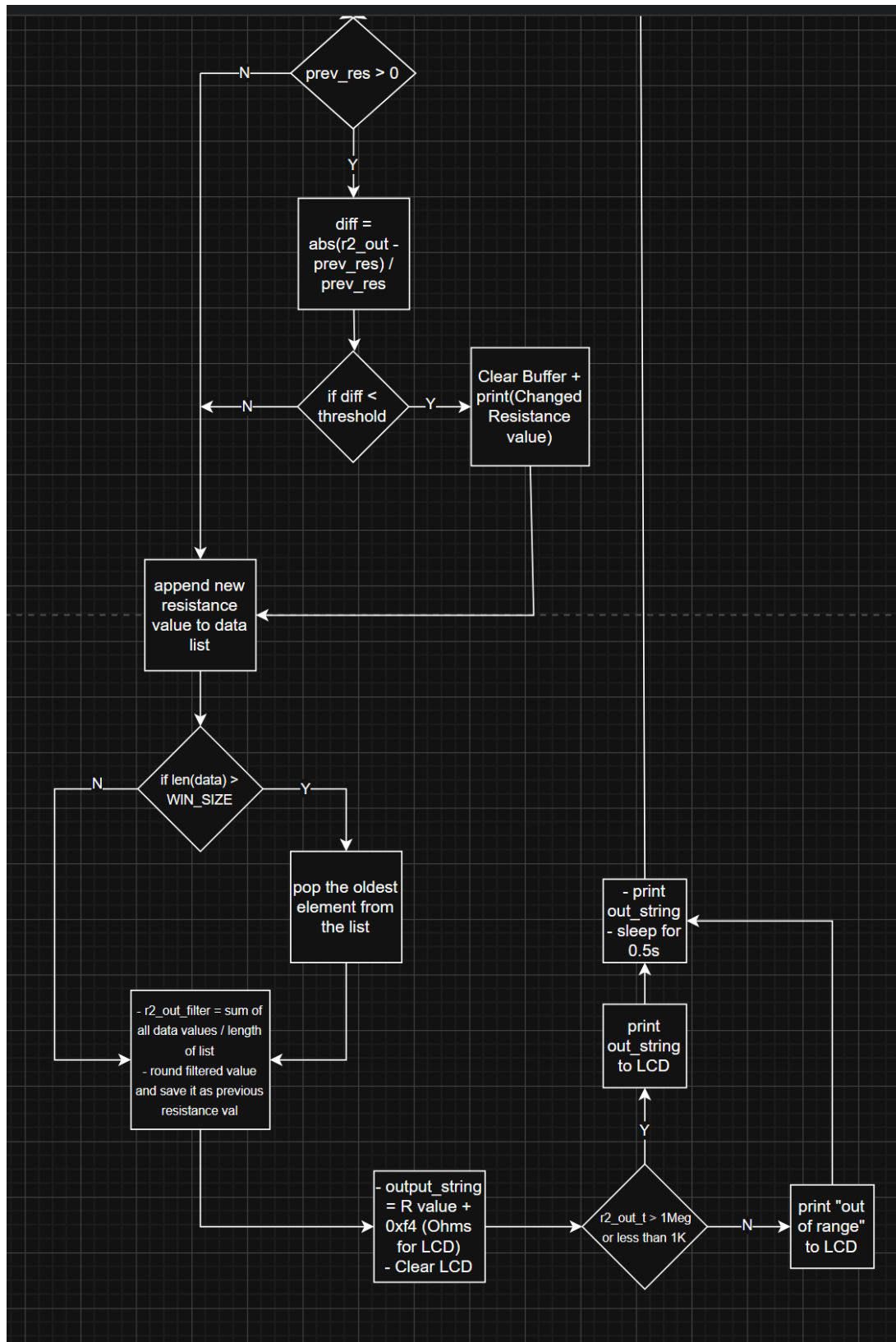


Figure 5: SW Block Bottom Half

Results

The results should be displayed in this section, which include figures and descriptive text for each figure. Figure captions need to be given as well, occurring below the figure. Use subheadings as necessary.

Figures needed:

- Picture of your LCD printing the resistance measurement.

Discuss the accuracy of your resistance measurements and any adjustments you needed to do to get a more accurate measurement.

Conclusion

The conclusion should reiterate key points learned in this module.

Appendix

```
import machine
import time
import math

from time import sleep
from LCD import LCD

# LCD Instantiation

# GPIO
lcd = LCD(enable_pin=18,      # Enable Pin, int
          reg_select_pin=16,  # Register Select, int
          data_pins=[19, 20, 21, 22]  # Data Pin numbers for the upper nibble.
          list[int]
          )

lcd.init()
lcd.clear()
```

```

# initialize ADC
voltage_adc = machine.ADC(26)
# 16-bits because micropython upscales the 12 bit value by padding the lower 4
bits with 0
conversion_factor = 3.3 / 65535
# resistor 1 value
r1 = 32978
# linearization equation
lin_eq = 8913 * math.exp(2.57) * 10**(-6)

# Rolling Average Filter
data = []
WIN_SIZE = 50
prev_filtered_resistance = 0
THRESHOLD = 0.05

while True:
    # read the voltage and convert to adc value
    voltage_read = voltage_adc.read_u16() * conversion_factor
    # find r2 value

    if(3.3 - voltage_read <= 0):
        r2 = 1e7
    else:
        r2 = r1 * ((voltage_read) / (3.3 - voltage_read))

    ## convert the value using linearized equation (used exponential) for normal
    values and
    if(r2 < 650e3):
        r2_out = -16.263 + 0.9866 * (r2) - 2e-7 * (r2)**2
    else:
        r2_out = 0.821 * r2 + 15.477e3

    if prev_filtered_resistance > 0:

        diff = abs(r2_out - prev_filtered_resistance) / prev_filtered_resistance

        if diff > THRESHOLD:
            data.clear()
            print("Resistance Changed!")

    data.append(r2_out)

```

```

    if len(data) > WIN_SIZE:
        # remove the oldest sample if the length of the array is greater than the
        window size
        data.pop(0)

    # take the mean of the data values and the length
    r2_out_filter = sum(data) / len(data)
    r2_out_t = round(r2_out_filter, 3)
    prev_filtered_resistance = r2_out_filter

    # output the adc value to LCD display
    out_string = "R = " + str(r2_out_t) + chr(0xf4)

    # clear before printing
    lcd.clear()

    if (r2_out_t < 1e3 or r2_out_t > 1e6):
        lcd.print("Out of Range")
    else:
        lcd.print(out_string)

    # print out resistance value
    print(out_string)

    sleep(0.5)

```

References

- [1] <https://www.electronics-tutorials.ws/combination/analogue-to-digital-converter.html>
- [2] <https://uk.rs-online.com/web/content/discovery/ideas-and-advice/rotary-switches-guide>
- [3]